

```
=====
FIL: 12_1.py
=====
```

```
MORSE_KOD = {
    "A": ".-",
    "B": "-...",
    "C": "-.-.",
    "D": "-..",
    "E": ".",
    "F": "..-.",
    "G": "---.",
    "H": "....",
    "I": "..",
    "J": ".---",
    "K": "-.-",
    "L": ".-..",
    "M": "--",
    "N": "-.",
    "O": "---",
    "P": ".-.-.",
    "Q": "--.-",
    "R": ".-.",
    "S": "...",
    "T": "-",
    "U": "..-",
    "V": "...-",
    "W": ".--",
    "X": "-.-.-",
    "Y": "-.-.-",
    "Z": "--..",
    "Å": ".--.-",
    "Ä": "-.-.-",
    "Ö": "---.",
    " ": "/"
}
```

```
# Omvänd dictionary för att kunna översätta tillbaka
OMVÄND_MORSE = {v: k for k, v in MORSE_KOD.items()}
```

```
text = input("Skriv texten som ska översättas till morsekod: \n")
```

```
morse_lista = []
for tecken in text.upper():
    if tecken in MORSE_KOD:
        morse_lista.append(MORSE_KOD[tecken])
```

```
morse_resultat = " ".join(morse_lista)
print("\nPå morsekod:\n" + morse_resultat)
```

```
# Översätt tillbaka från morsekod till text
```

```

tillbaka_lista = []
kod_element = morse_resultat.split(" ")

for kod in kod_element:
    if kod in OMVÄND_MORSE:
        tillbaka_lista.append(OMVÄND_MORSE[kod])

print("\nTillbaka till vanlig text:\n" + "".join(tillbaka_lista))

```

```

=====
FIL: 12_2.py
=====

```

```

romerska_tal = {
    'I': 1,
    'V': 5,
    'X': 10,
    'L': 50,
    'C': 100,
    'D': 500,
    'M': 1000
}

inmatning = input("Ange ett romerskt tal (t.ex. XIV, MCMXCIX): ").upper()

totalt = 0
for i in range(len(inmatning)):
    nuvarande = romerska_tal[inmatning[i]]

    # Om det nuvarande tecknet är mindre än nästa, drar vi av värdet
    if i + 1 < len(inmatning) and nuvarande < romerska_tal[inmatning[i + 1]]:
        totalt -= nuvarande
    else:
        totalt += nuvarande

print(f"Det romerska talet {inmatning} blir: {totalt}")

```

```

=====
FIL: 3_2.py
=====

```

```

import math

radie = float(input("Ange cirkelns radie: "))

if radie > 0:
    omkrets = 2 * math.pi * radie
    area = math.pi * (radie**2)

```

```

        print(f"Cirkelns omkrets: {omkrets:.2f}")
        print(f"Cirkelns area: {area:.2f}")
    else:
        print("Fel: Radien måste vara större än 0.")

```

```

=====
FIL: 3_3.py
=====

```

```

import math

a = float(input("Ange längden på sida a: "))
b = float(input("Ange längden på sida b: "))
vinkel_grader = float(input("Ange vinkeln mellan sidorna (i grader): "))

if a > 0 and b > 0 and 0 < vinkel_grader < 180:
    vinkel_rad = math.radians(vinkel_grader)
    c = math.sqrt(a**2 + b**2 - 2 * a * b * math.cos(vinkel_rad))

    s_a = round(a, 4) # pga floats så att 4.99999 == 5 t ex
    s_b = round(b, 4)
    s_c = round(c, 4)

    print(f"\nTredje sidan (c) är: {c:.2f}")

    if s_a == s_b == s_c:
        print("Resultat: Triangeln är LIKSIDIG (alla sidor är lika).")
    elif s_a == s_b or s_a == s_c or s_b == s_c:
        print("Resultat: Triangeln är LIKBENT (två sidor är lika).")
    else:
        print("Resultat: Triangeln är OLIKSIDIG (alla sidor är olika).")
else:
    print(
        "Fel: Sidorna måste vara större än 0 och vinkeln måste vara mellan 0 och
180 grader."
    )

```

```

=====
FIL: 4_2.py
=====

```

```

for i in range(1, 13):
    print(f"{i} {i**2} {i**3}")

```

```

=====
FIL: 4_3.py
=====

```

```

salary = 0.01
target = 10**7
days = 0
total_sum = 0

while total_sum < target:
    days += 1
    total_sum += salary
    salary *= 2

print(f"Det behövs {days} dagar tills man tjänat 10 miljoner kronor.")

```

```

=====
FIL: 4_5.py
=====

```

```

# Kommunens befolkning

# Vid början av 2022 var invånarantalet 26 000
# Antalet födda och avlidna under ett år uppskattas vara 0.7% resp. 0.6%
# Antalet inflyttade och utflyttade uppskattas 300 resp. 325 per år

year = int(
    input("Vilket år vill du veta invånarantalet? Ange ett årtal efter 2022: ")
)

diff = year - 2022
population = 26000

for i in range(diff):
    born = population * 0.007
    dead = population * 0.006
    population = population + born - dead - 25 # 325 utflyttande minus 300
    inflyttande = -25

# Avrundar till närmsta heltal vid utskrift
print(f"År {year} beräknas invånarantalet vara {round(population)} st.")

```

```

=====
FIL: 5_2.py
=====

```

```

from datetime import date

dagens_datum = str(date.today())

personens_födelsedag = input("Vad är din födelsedag? (yyyy-mm-dd) \n")

```

```
if personens_födelsedag == dagens_datum:
    print("Grattis!!!")
```

```
=====
FIL: 5_3.py
=====
```

```
peronnummer = input("Ange personnummer: \n")

pnr_rensat = peronnummer.replace("-", "").replace("+", "")

nast_sista_siffran = int(pnr_rensat[-2])

if nast_sista_siffran % 2 == 0:
    print(f"Personen är en kvinna.")
else:
    print(f"Personen är en man.")
```

```
=====
FIL: 5_4och5_5.py
=====
```

```
text = input("Skriv texten som ska översättas till rövarspråket: \n")

konsonanter = "bcd fghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXZ"
resultat = []

for tecken in text:
    if tecken in konsonanter:
        if tecken.isupper():
            resultat.append(tecken + "o" + tecken.lower())
        else:
            resultat.append(tecken + "o" + tecken)
    else:
        resultat.append(tecken)

rovarsprak = "".join(resultat)
print("på rövarspråk:")
print(rovarsprak)

tillbaka = []
i = 0

while i < len(rovarsprak):
    tecken = rovarsprak[i]

    # Kollar om vi har ett mönster av typen konsonant + o + samma konsonant
    if tecken in konsonanter and i + 2 < len(rovarsprak):
        mitten = rovarsprak[i + 1]
```

```

        sista = rovarsprak[i + 2]

        if mitten in "oO" and sista.lower() == tecken.lower():
            tillbaka.append(tecken)
            i += 3 # Hoppa över 'o' och den extra konsonanten
            continue

    tillbaka.append(tecken)
    i += 1

print("\nTillbaka till vanlig text:\n" + "".join(tillbaka))

```

```

=====
FIL: 6_3.py
=====

```

```

input_lista = input("Ange element till listan (separera med komma): ").split(",")
input_tuppel = input("Ange element till tuplen (separera med komma): ").split(",")

lista = [x.strip() for x in input_lista]
tuppel = tuple(x.strip() for x in input_tuppel)

lika = True

if len(lista) != len(tuppel):
    lika = False
else:
    for elem_lista, elem_tuppel in zip(lista, tuppel):
        if elem_lista != elem_tuppel:
            lika = False
            break

if lika:
    print("Listan och tuplen är LIKA.")
else:
    print("Listan och tuplen är OLIKA.")

```

```

=====
FIL: 6_4.py
=====

```

```

import random

# Simulerar 9 st uppmätta temperaturvärden (mellan 16.0 och 25.0 °C)
antal_matningar = 100
matvarden = [round(random.uniform(16.0, 25.0), 1) for _ in range(antal_matningar)]

sorterade_varden = sorted(matvarden)
n = len(sorterade_varden)

```

```

mitten = n // 2

if n % 2 != 0:
    # Udda antal mätvärden
    median = sorterade_varden[mitten]
else:
    # Jämnt antal mätvärden
    median = (sorterade_varden[mitten - 1] + sorterade_varden[mitten]) / 2

print(f"Uppmätta värden: {matvarden}")
print(f"Sorterade värden: {sorterade_varden}")
print(f"Median: {median}")

```

```

=====
FIL: 6_7.py
=====

```

```

def magisk_fyrkant():
    n = int(input("Ange storlek på matrisen (N x N): "))
    matris = []

    for i in range(n):
        rad_input = input(
            f"Ange element för rad {i+1} (separera med mellanslag): "
        )
        rad = [int(x) for x in rad_input.split()]

        if len(rad) != n:
            print(f"Fel: Raden måste innehålla exakt {n} tal.")
            return False, []

        matris.append(rad)

    # referenssumman från första raden
    target = sum(matris[0])

    # Kontrollera alla rader
    for rad in matris:
        if sum(rad) != target:
            return False, matris

    # Kontrollera alla kolumner
    for c in range(n):
        if sum(matris[r][c] for r in range(n)) != target:
            return False, matris

    # Kontrollera båda diagonalerna
    diagonal1 = sum(matris[i][i] for i in range(n))
    diagonal2 = sum(matris[i][n - 1 - i] for i in range(n))

```

```
    if diagonal1 != target or diagonal2 != target:  
        return False, matris
```

```
    return True, matris
```

```
# Användning
```

```
ar_magisk, matris = magisk_fyrkant()
```

```
if ar_magisk:
```

```
    print("\nDet är en magisk fyrkant!")
```

```
else:
```

```
    print("\nDet är INTE en magisk fyrkant.")
```

```
=====
```

```
FIL: 6_9.py
```

```
=====
```

```
import math
```

```
inmatning = input("Ange vektorns komponenter (separera med mellanslag): ")
```

```
vektor = [float(x) for x in inmatning.split()]
```

```
kvadratsumma = sum(x**2 for x in vektor)
```

```
langd = math.sqrt(kvadratsumma)
```

```
print(f"\nVektor: {vektor}")
```

```
print(f"Matematisk längd: {langd:.2f}")
```